DS5007— Natural Language Processing (Fall 2026)
Programming Assignment 1: Binary text classification (Sarcastic vs. Non-Sarcastic)

Role

You are an NLP engineer building a binary sarcasm detector for Urdu social-media text. Your system must predict whether each post is sarcastic (1) or non-sarcastic (0).

Size (for reference): ~20k rows (a few rows may have missing labels — drop these).

Urdu Text	Is_scaristic
یہ اسمبلی بغیر ڈیزل کے چل رہی ہے کیونکہ اِسکو گدھا " چلا رہا ھے گدھے کو ڈیزل کی نہیں `` ڈنڈے " کی ضرورت ہوتی ہے 😂😂😂	1
ویسے اپنے پیر پر کلہاڑی مارنا کوئ ان سے سیکھے 😂	1
ان کو لگتا ہے کہ انکے دیوتا کی طرح پورا پاکستان `` لکیروں " کے سہارے سوچتا اور بولتا ہے۔۔۔😂😂😂	0

Input column: urdu_text
Target column: is_sarcastic, where 1 = sarcastic and 0 = non-sarcastic.
The text may contain Urdu script, Roman/English tokens, emojis, punctuation, hashtags, mentions, URLs, spelling variation, and informal social-media writing.
Inspect missing values, duplicates, label balance, and text length before modelling. Use a stratified train/validation/test split and avoid test-set leakage.

Goal

1. Exploratory analysis - report class distribution, text-length statistics, representative examples, and common data-quality issues.
2. Urdu-aware preprocessing - design and justify normalization/tokenization. Do not blindly remove emojis, punctuation, repetition, hashtags, or code-mixed terms; test whether these are useful sarcasm cues.
3. Classical baseline - train at least one TF-IDF based classifier (e.g., Logistic Regression, Linear SVM, or Naive Bayes). Word and/or character n-grams are allowed.
4. Neural model - train at least one neural classifier (e.g., CNN, BiLSTM/GRU, or another justified architecture) using Keras/TensorFlow or PyTorch.
5. Transformer experiment - fine-tune an Urdu-capable or multilingual pretrained transformer (e.g., an appropriate BERT/XLM-R family model) and compare it with the baseline and neural model.
6. Ablation study - evaluate at least two preprocessing/feature choices, such as keeping vs. removing emojis, word vs. character features, or normalized vs. minimally processed text.

Evaluation

Report Accuracy, Precision, Recall, F1 for each class, Macro-F1, and a confusion matrix. Use Macro-F1 as the primary model-comparison metric.
Also include error analysis of at least 10 misclassified examples, covering linguistic phenomena such as irony/context dependence, emojis, negation, idioms, spelling variation, code-mixing, and ambiguous labels where applicable.

Deliverables (what you submit)

A Jupyter notebook (or .py) with clean and runnable code. You can extend the provided notebook and use it as a skeleton code.

A short report (2–3 pages PDF) including: data prep, models, features, experiments, results table, confusion matrices, and error analysis (at least 5 misclassified examples with your commentary). You can also use your notebook to provide these within the notebook.

Optional: saved model weights and vectorizers/tokenizers if reproducibility requires them. You can share your GitHub repo for reproducibility

Graduate-level work should explain why each modelling and preprocessing choice is appropriate for Urdu sarcasm rather than only reporting code output. Any external code, pretrained model, dataset source, or AI-assisted material must be properly acknowledged according to course policy.

Bonus marks – for feature engineering of Urdu text features like phrases etc.

Due Date- October 15, 2026 at midnight